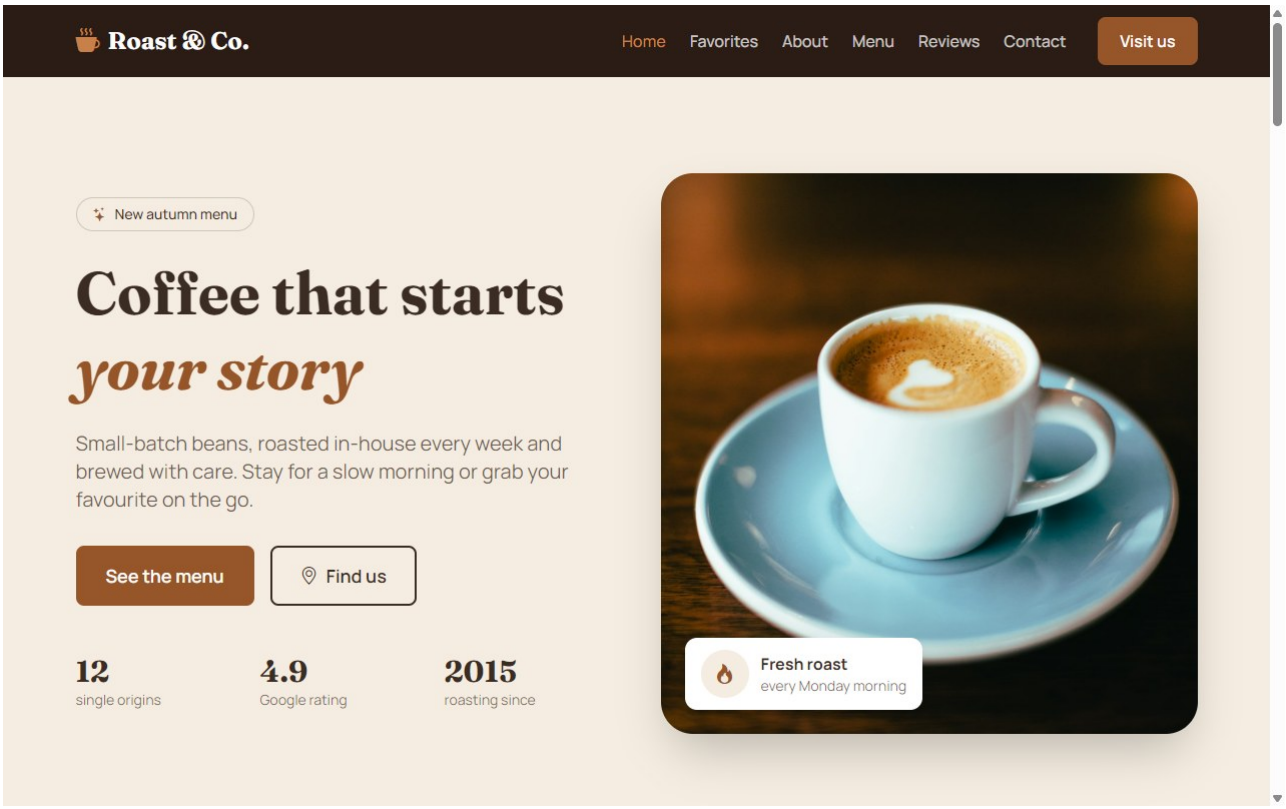
MODULE 2 · FRONTEND FUNDAMENTALS & TOOLING

Eén app die groeit

Roast & Co. – van landingspagina tot webapp



Naam: _____

Klas: _____

Inhoud

Inleiding **4**

Stap 1 · Tailwind CSS I **7**

1.1 · Het startproject 8
1.2 · De navbar 12
1.3 · De hero 16
1.4 · Populaire koffies 22
1.5 · Oefening 1 — Het startproject van Sel & Poivre 28
1.6 · Oefening 2 — Navbar en hero 29
1.7 · Challenge — Binnenkort in het atelier 30

Stap 2 · Tailwind CSS II **32**

2.1 · Over Roast & Co. 32
2.2 · De menukaart 36
2.3 · Klantenbeoordelingen 43
2.4 · Call-to-action 47
2.5 · Het contactformulier 51
2.6 · Footer en eindcontrole 58
2.7 · Oefening 1 — Over het atelier 62
2.8 · Oefening 2 — Het programma 63
2.9 · Oefening 3 — Inschrijven en footer 65
2.10 · Challenge — Reviews en call-to-action 66

Stap 3 · Data in JavaScript **69**

3.1 · Een product als object 70
3.2 · Loops 74
3.3 · Arraymethoden 77
3.4 · De kaart tekenen en debuggen 80
3.5 · Oefening 1 — Het programma verkennen 86
3.6 · Oefening 2 — Het programma uit de gegevens 87
3.7 · Challenge — Samenvattingen 88

Stap 4 · DOM en events **90**

4.1 · De DOM en het hamburgermenu 91
4.2 · Events en de tabs 94
4.3 · Elementen maken: het mandje 96
4.4 · Formulieren, JSON en localStorage 103
4.5 · Oefening 1 — Menu en tabs 108
4.6 · Oefening 2 — De verlanglijst 109
4.7 · Oefening 3 — Inschrijven 110
4.8 · Challenge — Zoeken 111

Stap 5 · Modules en fetch **113**

5.1 · Van scripts naar modules 115
5.2 · Een API verkennen met Postman 121
5.3 · De kaart van de API 126
5.4 · Bestellen met POST 133

5.5 · Oefening 1 — Sel & Poivre in modules 140
5.6 · Oefening 2 — Het programma van de API 141
5.7 · Oefening 3 — Inschrijven via de API 143
5.8 · Challenge — Mijn inschrijvingen 144

Stap 6 · Node, npm en Vite 146

6.1 · npm en package.json 147
6.2 · Tailwind en de iconen als pakket 150
6.3 · Bouwen en onderhouden 154
6.4 · Oefening 1 — Sel & Poivre op npm en Vite 157
6.5 · Oefening 2 — Tailwind als pakket en een build 157
6.6 · Challenge — Eén commando 158

Stap 7 · TypeScript 160

7.1 · TypeScript in het project 161
7.2 · Types voor je gegevens 167
7.3 · Gegevens van buiten: unknown en een generic 173
7.4 · Oefening 1 — TypeScript in Sel & Poivre 181
7.5 · Oefening 2 — Types voor het programma 181
7.6 · Challenge — Alles getypeerd 182

Stap 8 · React I 184

8.1 · React in het project 185
8.2 · Componenten en props 190
8.3 · Events en useState 193
8.4 · Oefening 1 — Het programma in React 197
8.5 · Oefening 2 — De seizoenen als tabs 198
8.6 · Challenge — Zoeken in React 199

Stap 9 · React II 201

9.1 · Gegevens ophalen met useEffect 202
9.2 · Een eigen hook 208
9.3 · State optillen: het mandje en bestellen in React 212
9.4 · Oefening 1 — Het programma laadt zelf 221
9.5 · Oefening 2 — De verlanglijst in React 222
9.6 · Challenge — Mijn inschrijvingen in React 223

Stap 10 · Globale state met Zustand 225

10.1 · Een store voor het mandje 226
10.2 · Bewaren met persist en een tweede root 232
10.3 · Bestellen als async actie 235
10.4 · Oefening 1 — De verlanglijst in een store 240
10.5 · Oefening 2 — Mijn inschrijvingen in een store 241
10.6 · Challenge — Het programma in een store 243

Stap 11 · Toegankelijkheid 245

11.1 · Meten met axe 246
11.2 · Toetsenbord en focus 249
11.3 · De schermlezer en ARIA 254
11.4 · Oefening 1 — Sel & Poivre gemeten 258

11.5 · Oefening 2 — Wat de schermlezer hoort 259
11.6 · Challenge — Mijn inschrijvingen zonder muis 260

Oplossingen 261

Oplossing 1.5 · Het startproject van Sel & Poivre 261
Oplossing 1.6 · Navbar en hero 263
Oplossing 1.7 · Binnenkort in het atelier 267
Oplossing 2.7 · Over het atelier 270
Oplossing 2.8 · Het programma 272
Oplossing 2.9 · Inschrijven en footer 276
Oplossing 2.10 · Reviews en call-to-action 280
Oplossing 3.5 · Het programma verkennen 283
Oplossing 3.6 · Het programma uit de gegevens 285
Oplossing 3.7 · Samenvattingen 288
Oplossing 4.5 · Menu en tabs 290
Oplossing 4.6 · De verlanglijst 291
Oplossing 4.7 · Inschrijven 295
Oplossing 4.8 · Zoeken 297
Oplossing 5.5 · Sel & Poivre in modules 300
Oplossing 5.6 · Het programma van de API 303
Oplossing 5.7 · Inschrijven via de API 308
Oplossing 5.8 · Mijn inschrijvingen 311
Oplossing 6.4 · Sel & Poivre op npm en Vite 316
Oplossing 6.5 · Tailwind als pakket en een build 316
Oplossing 6.6 · Eén commando 320
Oplossing 7.4 · TypeScript in Sel & Poivre 321
Oplossing 7.5 · Types voor het programma 325
Oplossing 7.6 · Alles getypeerd 329
Oplossing 8.4 · Het programma in React 336
Oplossing 8.5 · De seizoenen als tabs 340
Oplossing 8.6 · Zoeken in React 343
Oplossing 9.4 · Het programma laadt zelf 346
Oplossing 9.5 · De verlanglijst in React 352
Oplossing 9.6 · Mijn inschrijvingen in React 357
Oplossing 10.4 · De verlanglijst in een store 362
Oplossing 10.5 · Mijn inschrijvingen in een store 366
Oplossing 10.6 · Het programma in een store 371
Oplossing 11.4 · Sel & Poivre gemeten 375
Oplossing 11.5 · Wat de schermlezer hoort 379
Oplossing 11.6 · Mijn inschrijvingen zonder muis 380

Inleiding

Hoe werkt deze module?

Eén app die groeit

In module 1 bouwde je de landingspagina van **Roast & Co.** met Bootstrap, en je zette je eerste stappen in JavaScript. In deze module bouw je op diezelfde koffiebar verder, tot het een echte webapp is waarin een klant de kaart bekijkt, een mandje vult en een bestelling verstuurt.

Je bouwt die app niet in één keer. Elke stap voegt één laag toe, en elke laag lost een probleem op dat je in de vorige stap zelf tegenkomt.

Stap	Onderwerp	Roast & Co. na deze stap
1	Tailwind CSS I	navbar, hero en kaarten in een eigen huisstijl
2	Tailwind CSS II	de volledige landingspagina, met formulier en footer
3	Data in JavaScript	de koffiekaart komt uit een lijst van objecten
4	DOM en events	de pagina reageert: menu, tabs, mandje, formulier
5	Modules en fetch	de kaart komt van een API, een bestelling gaat naar de server
6	Node, npm en Vite	het project wordt een echt project met een buildstap
7	TypeScript	de code controleert zichzelf voor ze draait
8	React I	de kaart bestaat uit React-componenten
9	React II	bestellen werkt volledig in React
10	Globale state met Zustand	het mandje zit in een store, bestellen is een async actie, en alles blijft bewaard
11	Toegankelijkheid	de app werkt met het toetsenbord en de schermlezer, en doorstaat een audit

Deze bundel bevat alle elf stappen. Het twaalfde lesmoment is het examen.

Twee apps: samen en alleen

Elke stap heeft twee delen.

1. **Samen bouwen.** De docent bouwt Roast & Co. stap voor stap op het scherm, en jij typt mee op je eigen computer. Elke paragraaf eindigt met een werkend resultaat dat je in de browser controleert.
2. **Zelf oefenen.** Daarna pas je dezelfde leerstof toe op een **tweede app**: **Sel & Poivre**, een kookatelier in Gent dat workshops organiseert. Die app groeit mee, stap na stap. Aan het einde van de module heb je dus twee volledige apps gebouwd: één samen, en één zelf.

De oefeningen worden telkens moeilijker. **Oefening 1** past de leerstof rechtstreeks toe. **Oefening 2** combineert meerdere onderdelen, met minder uitleg. De **challenge** vraagt eigen keuzes en is bedoeld voor wie sneller klaar is.

> **Goed om te weten**
>
> Sel & Poivre is niet toevallig gekozen. Het kookatelier zit in de oefen-API die je vanaf stap 5 gebruikt: de workshops, de chefs en de prijzen op je landingspagina zijn dezelfde als die de API straks teruggeeft.

Startpunt en oplossing bij elke paragraaf

Naast deze bundel staan twee mappen, die je strikt uit elkaar houdt:

```
module-02/
├── Module-02-Roast-en-Co.docx            ← deze bundel
├── 1-begeleid-Roast-and-Co/              ← wat we SAMEN bouwen
│   ├── afbeeldingen/
│   └── stap-01-Tailwind CSS I/
│       ├── 1.01-Het startproject/
│       │   ├── startpunt/                ← zo begin je aan 1.1
│       │   └── oplossing/roast-and-co/   ← zo ziet het eruit na 1.1
│       └── 1.02-De navbar/ …
└── 2-oefeningen-Sel-et-Poivre/           ← wat je ALLEEN bouwt
    ├── afbeeldingen/
    └── stap-01-Tailwind CSS I/
        └── 1.05-Oefening 1 - Het startproject van Sel & Poivre/
            ├── opgave.md
            ├── startpunt/
            └── oplossing/sel-et-poivre/
```

- Het **startpunt** van een paragraaf is altijd de **oplossing** van de paragraaf ervoor. Raakte je achterop, of loopt er iets mis dat je niet meteen vindt? Kopieer de projectmap uit `startpunt/` en werk vandaar verder.
- De **oplossing** gebruik je om te vergelijken, niet om over te nemen. Bij de oefeningen kijk je er pas in als je zelf klaar bent of echt vastzit.
- Onder elke paragraaf in deze bundel staat in welke map je startpunt en oplossing vindt.

Wat je nodig hebt

- **Visual Studio Code** met de extensie **Live Server** — allebei geïnstalleerd in module 1.
- **Chrome** of **Edge**, met de DevTools (`F12`).
- Een **internetverbinding**: Tailwind, de iconen en de lettertypes komen in stap 1 en 2 van een CDN.
- **Git**: je maakt van beide apps een repository en commit na elke paragraaf. Dat ken je uit module 1.

Zo lees je deze bundel

Een **gewoon codeblok** typ (of kopieer) je volledig over:

```
<div class="mx-auto max-w-6xl px-4">
  <h1 class="text-4xl font-bold">Hello!</h1>
</div>
```

Pas je **bestaande code** aan, dan zie je een blok met `+` en `-` vooraan de regels:

```
  <div class="mx-auto max-w-6xl px-4">
-   <h1 class="text-4xl font-bold">Hello!</h1>
+   <h1 class="font-display text-5xl font-bold">Hello!</h1>
  </div>
```

- Regels met `+` (groen) voeg je **toe**; regels met `-` (rood) **verwijder** je.
- Regels zonder teken staan er al: ze tonen **waar** je werkt.
- Het `+`- of `-`-teken zelf typ je niet over. `…` betekent: hier staat code die niet verandert.

De gekleurde kaders betekenen:

> **Tip**
>
> Een handige tip of snelkoppeling.

> **Goed om te weten**
>
> Achtergrond: waarom werkt iets zo?

> **Let op**
>
> Een veelgemaakte fout. Lees dit zeker.

> **Weglaten of vervangen**
>
> Hier laat je code **weg** of **vervang** je ze.

> **Zelf doen**
>
> Een korte opdracht om zelf uit te proberen.

De uitgewerkte oplossingen van de oefeningen staan **achteraan** deze bundel, onder *Oplossingen*.

Stap 1 · Tailwind CSS I

Utility-first werken: het thema, de navbar, de hero en een grid met kaarten

Wat bouwen we?

In module 1 bouwde je de landingspagina van Roast & Co. met **Bootstrap**: kant-en-klare componenten zoals `navbar`, `card` en `btn`, die je met een paar classes in elkaar klikte. In deze stap begin je opnieuw aan dezelfde pagina, maar nu met **Tailwind CSS 4**.

Tailwind werkt helemaal anders. Je krijgt **geen componenten**, maar duizenden kleine classes die elk **één ding** doen: `p-4` zet padding, `text-xl` de lettergrootte, `bg-white` de achtergrond. Door ze te combineren bouw je elk ontwerp zelf op, rechtstreeks in je HTML. Je eigen kleuren en lettertypes leg je vast in een **thema**.

Na deze stap staat het bovenste deel van de pagina er: de navigatiebalk, de hero en drie productkaarten.

Paragraaf	Wat je bouwt	Wat je leert
1.1	Het startproject	Play CDN, `@theme`, utility-classes, Preflight
1.2	De navbar	flexbox met utilities, breakpoints, `hidden`/`lg:flex`, `@apply`
1.3	De hero	CSS grid, de spacing-schaal, arbitrary values, `position`
1.4	Populaire koffies	een responsive grid, kaarten, `aspect-*`, `group-hover`, transities
1.5–1.7	Sel & Poivre	dezelfde leerstof, toegepast op je eigen app

Wat je gebruikt uit de vorige modules

Je gebruikt	Waar je het leerde
Een HTML-pagina met semantische structuur: `header`, `nav`, `main`, `section`, `article`	module 1, lesdag 3
Headinghiërarchie: één `h1`, daaronder `h2` en `h3`	module 1, lesdag 3
`alt`, `aria-label`, `aria-hidden`	module 1, lesdag 4
Selectors, het boxmodel, kleur en contrast	module 1, lesdag 5
Mobile first, breakpoints, het idee van een rooster	module 1, lesdag 6 (Bootstrap)
Live Server, de DevTools en de responsive modus	module 1, lesdag 1
Commits en een repository op GitHub	module 1, lesdag 2

> **Goed om te weten**
>
> Je Bootstrap-versie van Roast & Co. heet waarschijnlijk ook `roast-and-co`. Hernoem die map naar `roast-and-co-bootstrap` voor je begint. Dan kan je de twee versies naast elkaar openen en vergelijken.

Bootstrap en Tailwind naast elkaar

Bootstrap	Tailwind
Kant-en-klare componenten (`card`, `navbar`, `btn`)	Je bouwt alles zelf met kleine utilities
`container` › `row` › `col-lg-6`	`mx-auto max-w-6xl` › `grid lg:grid-cols-2`
Breakpoints `md` = 768px, `lg` = 992px	Breakpoints `md` = 768px, `lg` = **1024px**
JavaScript voor menu, tabs, … inbegrepen	Geen JavaScript: dat schrijf je zelf (vanaf stap 4)
Aanpassen via `--bs-`-variabelen	Aanpassen via `@theme`

1.1 · Het startproject

In deze paragraaf zet je het project klaar en test je of alles correct gekoppeld is.

a · De map

Maak een nieuwe map `roast-and-co` met deze inhoud:

```
roast-and-co/
├── index.html
└── img/          (de foto's komen vanaf 1.3)
```

Er is **geen CSS-bestand**. Met de Play CDN van Tailwind schrijf je je eigen CSS in een `<style>`-blok in de `<head>` (zie d).

Open de map in VS Code, maak er een Git-repository van en zet `index.html` erin:

```
git init
```

b · Het HTML-skelet

Typ in het lege `index.html` een `!` en druk op `Tab`. VS Code maakt een basisskelet. Pas het aan tot dit:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Roast &amp;amp; Co. – artisan coffee for every moment. Small-batch coffee,
roasted in Gent.">
  <title>Roast &amp;amp; Co. – artisan coffee for every moment</title>
</head>
<body>


</body>
</html>
```

`lang="en"`: de website van Roast & Co. is Engelstalig, net als in module 1. De `viewport`-regel ken je: zonder die regel toont een smartphone een verkleinde desktopversie.

c · Tailwind, iconen en lettertypes koppelen

De eenvoudigste manier om Tailwind te gebruiken is de **Play CDN**: één `<script>` in de `<head>`. Daaronder koppel je **Bootstrap Icons** (een losse icoonbibliotheek die ook zonder Bootstrap werkt) en twee lettertypes van **Google Fonts**: *Fraunces* voor titels en *Manrope* voor tekst.

```
    <title>Roast &amp;amp; Co. – artisan coffee for every moment</title>
+
+   <!-- Tailwind CSS 4.3.3 (Play CDN) -->
+   <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4.3.3"></script>
+   <!-- Bootstrap Icons -->
+   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-
icons.min.css">
+   <!-- Google Fonts: Fraunces (titels) en Manrope (tekst) -->
+   <link rel="preconnect" href="https://fonts.googleapis.com">
+   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
+   <link
href="https://fonts.googleapis.com/css2?family=Fraunces:ital,opsz,wght@0,9..144,600;0,9..144,700;1,9..144,600
;1,9..144,700&family=Manrope:wght@400;500;600;700&display=swap" rel="stylesheet">
  </head>
```

> **Goed om te weten**
>
> **Hoe werkt de Play CDN?** Het script leest in je browser welke classes je in je HTML gebruikt en maakt **alleen voor die classes** de CSS aan. Schrijf je `p-4`, dan bestaat de regel `padding: 1rem`. Gebruik je een class nergens, dan wordt ze ook niet gemaakt.

> **Let op**
>
> De Play CDN is bedoeld om te **leren en te ontwikkelen**. Voor een echte website laat je Tailwind op voorhand één CSS-bestand maken. Dat doe je in stap 6, wanneer het project op Vite overgaat.

Een icoon gebruik je zo: `<i class="bi bi-cup-hot-fill"></i>`. Zoek iconen op icons.getbootstrap.com.

d · De huisstijl met @theme

Tailwind heeft een groot standaardpalet (`red-500`, `sky-300` …), maar Roast & Co. heeft **eigen merkkleuren**. Die leg je vast in een `@theme`-blok, binnen een `<style type="text/tailwindcss">` onderaan de `<head>`:

```
    <link href="https://fonts.googleapis.com/css2?family=..." rel="stylesheet">
+   <!-- Eigen thema en componenten -->
+   <style type="text/tailwindcss">
+     /* hier komt de code hieronder */
+   </style>
  </head>

/* Eigen huisstijl: elke variabele in @theme wordt een reeks utility-classes */
@theme {
  --color-espresso: #2b1d16;
  --color-caramel: #c8814a;
  --color-caramel-dark: #97562a;
  --color-cream: #f5ede1;
  --color-foam: #fcf8f2;
  --color-ink: #3b2c24;

  --font-display: "Fraunces", Georgia, serif;
  --font-sans: "Manrope", system-ui, sans-serif;
}
```

Wat gebeurt hier?

- Elke variabele die begint met `--color-` wordt een kleur die je in **alle** kleurclasses kan gebruiken. Uit `--color-espresso` maakt Tailwind automatisch `bg-espresso`, `text-espresso`, `border-espresso` …
- `--font-display` wordt de class `font-display`. `--font-sans` overschrijft het standaardlettertype, dus alle tekst wordt meteen Manrope.
- `type="text/tailwindcss"` is nodig: zo weet de Play CDN dat hij dit blok moet verwerken.

e · De body en een testblok

Geef de `<body>` meteen de basisstijl en zet tijdelijk een testblok in een `<main>`:

```
<div class="mx-auto max-w-6xl px-4 py-12">
  <h1 class="font-display text-4xl font-bold">Hello, Roast &amp;amp; Co.!</h1>
  <p class="mt-3 text-lg text-ink/75">If this heading uses a serif font and the button below is caramel, everything is linked correctly.</p>
  <button type="button" class="mt-6 rounded-lg bg-caramel-dark px-5 py-2.5 font-semibold text-white hover:bg-espresso">Test button</button>
</div>

<body class="bg-foam font-sans text-ink antialiased">
  <main>
    <!-- het testblok hierboven -->
  </main>
</body>
```

Class	Wat doet het?
`bg-foam`, `text-ink`	Je eigen achtergrond- en tekstkleur uit `@theme`.
`antialiased`	Iets fijnere letterweergave.
`mx-auto max-w-6xl px-4`	**Het containerpatroon:** maximaal 72rem breed, gecentreerd, met ruimte aan de zijkanten. Dit gebruik je in elke sectie. Het vervangt de `container` van Bootstrap.
`py-12`	Padding boven en onder: 12 × 0.25rem = 3rem.
`text-4xl`, `text-lg`	Lettergrootte.
`font-bold`, `font-semibold`	Letterdikte.
`text-ink/75`	Tekstkleur *ink* met **75 % dekking**. Die `/getal` werkt op elke kleur.
`mt-3`, `mt-6`	Margin-top.
`rounded-lg`	Afgeronde hoeken.
`hover:bg-espresso`	Een **variant**: deze class werkt alleen bij hover.

> **Goed om te weten**
>
> **Waarom is `<h1>` niet vanzelf groot?** Tailwind zet via *Preflight* alle standaardopmaak van de browser op nul: titels, marges, lijstbolletjes. Je begint met een lege lei en bepaalt alles met classes.

Wat er wijzigt, en waar

Alles wat in deze paragraaf in de code verandert, per bestand en met de plaats waar het komt. Een `+` voeg je toe, een `-` haal je weg; de regels zonder teken staan er al en tonen waar je zit. De regelnummers zijn die van het bestand ná de wijziging.

`index.html` · nieuw bestand; het staat volledig hierboven.

Controleer

Open `index.html` met Live Server.

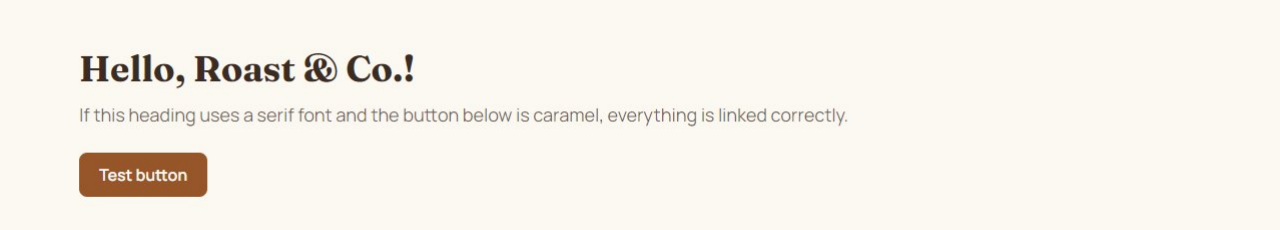
- De titel staat in een **schreefletter** (Fraunces): Google Fonts en `font-display` werken.
- De knop is **karamelbruin** en wordt donker bij hover: `@theme` werkt.
- De achtergrond is licht crèmekleurig in plaats van wit.

Maak je eerste commit: `git add .` en `git commit -m "Startproject met Tailwind en eigen thema"`.

> **Zelf doen**
>
> Verander in `@theme` de waarde van `--color-caramel-dark` in `#1d4ed8` en sla op. Wat gebeurt er met de knop? Zet de waarde daarna terug. Je ziet hier het grote voordeel van een thema: één waarde aanpassen verandert elke plaats waar de kleur gebruikt wordt.

Documentatie

- Tailwind: Play CDN
- Tailwind: Theme variables (@theme)
- Tailwind: Styling with utility classes
- Tailwind: Preflight



Resultaat na 1.1

> **Goed om te weten**
>
> **Startpunt en oplossing van 1.1:** `1-begeleid-Roast-and-Co/stap-01-Tailwind CSS I/1.01-Het startproject/`, met daarin `startpunt/` en `oplossing/`.

1.2 · De navbar

Een **navigatiebalk** die bovenaan blijft plakken. Op een groot scherm staan de links naast elkaar; op een smartphone staat er een **hamburgerknop**.

In Bootstrap kreeg je de navbar kant-en-klaar, inclusief het JavaScript om het menu open te klappen. Tailwind heeft geen navbar en ook geen JavaScript. Je bouwt de balk dus zelf, met utilities.

> **Tip**
>
> Bekijk je pagina op smartphonebreedte: druk op `F12` en daarna op `Ctrl` + `Shift` + `M` (responsive modus).

a · Het testblok weghalen

> **Weglaten of vervangen**
>
> Het testblok uit 1.1 heeft zijn werk gedaan. Haal de inhoud van `<main>` weg; het `<main>`-element zelf blijft staan.

```
  <body class="bg-foam font-sans text-ink antialiased">
    <main>
-     <div class="mx-auto max-w-6xl px-4 py-12">
-       <h1 class="font-display text-4xl font-bold">Hello, Roast &amp;amp; Co.!</h1>
-       ...
-     </div>
+     <!-- Hier komen straks de secties -->
    </main>
```

b · Een balk met flexbox

In Bootstrap zette je `d-flex` op een element om de kinderen naast elkaar te zetten. In Tailwind heet die class gewoon `flex`. Zet dit **boven** `<main>`:

```
<header class="sticky top-0 z-50 bg-espresso text-white">
  <nav id="mainNav" class="mx-auto flex max-w-6xl flex-wrap items-center justify-between px-4 py-3" aria-
label="Main navigation">
    <a href="#home" class="flex items-center gap-2 font-display text-2xl font-bold">
      <i class="bi bi-cup-hot-fill text-caramel" aria-hidden="true"></i>
      Roast &amp;amp; Co.
    </a>
    <!-- knop (c) en menu (d) -->
  </nav>
</header>
```

Class	Wat doet het?
`sticky top-0`	De balk plakt bovenaan tijdens het scrollen.
`z-50`	Ligt boven de rest van de pagina (*z-index*).
`flex`	Een **flexbox**: de kinderen (logo, knop, menu) staan naast elkaar.
`flex-wrap`	Te weinig plaats? Dan springt het menu naar een nieuwe regel.
`items-center`	Verticaal centreren in de rij.
`justify-between`	Het eerste kind helemaal links, het laatste helemaal rechts, de ruimte ertussen.

Class	Wat doet het?
`gap-2`	Ruimte tussen de kinderen: tussen icoon en merknaam.

> **Goed om te weten**
>
> **Flexbox in één zin.** Een element met `flex` zet zijn kinderen op één lijn: standaard naast elkaar (`flex-row`), of onder elkaar met `flex-col`. Met `justify-*` verdeel je ze over de lange as, met `items-*` lijn je ze uit op de korte as. Dat is alles wat je in deze stap nodig hebt.

c · Breakpoints: mobile first

Net als Bootstrap is Tailwind **mobile first**: een class zonder voorvoegsel geldt op **elk** scherm. Met een voorvoegsel zoals `lg:` geldt ze pas **vanaf** die breedte.

Voorvoegsel	Vanaf	Typisch toestel
(geen)	0px	smartphone
`sm:`	640px	grote smartphone
`md:`	768px	tablet
`lg:`	1024px	laptop
`xl:`	1280px	desktop

De hamburgerknop is alleen nodig op kleine schermen: zichtbaar, en **verborgen vanaf** `lg`. Zet hem op de plaats van de commentaarregel:

```
<button id="navToggle" type="button" class="rounded-lg border border-white/30 px-3 py-1 text-2xl lg:hidden"
aria-controls="navMenu" aria-expanded="false" aria-label="Toggle navigation">
  <i class="bi bi-list" aria-hidden="true"></i>
</button>
```

De knop heeft geen tekst, alleen een icoon. Daarom krijgt hij een `aria-label`, zoals je in module 1 leerde.

d · Het menu

Het menu doet het omgekeerde: **verborgen** op mobiel, **zichtbaar als flexbox** vanaf `lg`.

```
<div id="navMenu" class="hidden w-full lg:flex lg:w-auto lg:items-center lg:gap-8">
  <ul class="flex flex-col py-4 font-medium lg:flex-row lg:gap-6 lg:py-0">
    <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel" href="#home"
data-active>Home</a></li>
    <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#favorites">Favorites</a></li>
    <!-- About, Menu, Reviews, Contact op dezelfde manier -->
  </ul>
  <a class="btn btn-roast mb-4 lg:mb-0" href="#visit">Visit us</a>
</div>
```

Class	Wat doet het?
`hidden lg:flex`	Verborgen, en vanaf `lg` een flexbox.
`w-full lg:w-auto`	Op mobiel volle breedte (onder het logo), op een laptop zo breed als nodig.

Class	Wat doet het?
`flex-col lg:flex-row`	Links **onder** elkaar op mobiel, **naast** elkaar vanaf `lg`.
`text-white/75`	Wit met 75 % dekking; `hover:text-white` maakt het volledig wit.
`transition`	De kleurwissel verloopt vloeiend.
`data-active:text-caramel`	Heeft de link het attribuut `data-active`, dan wordt hij karamel. Voorlopig staat dat vast op *Home*; in stap 4 verplaatst JavaScript het.

> ⚠ **Let op**
>
> Op een smartphone zie je nu alleen het logo en de hamburgerknop, en de knop doet nog niets. Dat is voorlopig zo: een knop laten reageren op een klik vraagt JavaScript dat je pas in stap 4 schrijft. De `id`'s `navToggle` en `navMenu` en het attribuut `aria-expanded` staan er al klaar voor.

e · Een herbruikbare knop met @apply

De knop *Visit us* krijgt de classes `btn btn-roast`. Die bestaan niet in Tailwind: je maakt ze zelf, want deze knopstijl komt nog vaak terug. Met `@apply` bundel je utilities onder één eigen classnaam. Voeg dit toe in het `<style type="text/tailwindcss">`-blok, onder `@theme`:

```
/* Knoppen die we vaak hergebruiken: een component met @apply */
@layer components {
  .btn {
    @apply inline-flex items-center justify-center gap-2 rounded-lg border-2 px-5 py-2.5 font-semibold
transition;
  }

  .btn-roast {
    @apply border-caramel-dark bg-caramel-dark text-white hover:border-espresso hover:bg-espresso;
  }
}
```

- `@layer components` zet je eigen classes in de laag **vóór** de utilities. Daardoor kan je ze altijd nog aanpassen met een gewone utility: `btn btn-roast mb-4` werkt zoals je verwacht.
- Binnen `@apply` mag je ook varianten gebruiken, zoals `hover:bg-espresso`.

> ⚠ **Let op**
>
> Gebruik `@apply` met mate. De kracht van Tailwind is dat je in de HTML ziet hoe iets eruitziet. Bundel alleen wat je echt vaak herhaalt, zoals knoppen.

De volledige navbar

```
<header class="sticky top-0 z-50 bg-espresso text-white">
  <nav id="mainNav" class="mx-auto flex max-w-6xl flex-wrap items-center justify-between px-4 py-3" aria-
label="Main navigation">
    <a href="#home" class="flex items-center gap-2 font-display text-2xl font-bold">
      <i class="bi bi-cup-hot-fill text-caramel" aria-hidden="true"></i>
      Roast &amp;amp; Co.
    </a>
    <button id="navToggle" type="button" class="rounded-lg border border-white/30 px-3 py-1 text-2xl
lg:hidden" aria-controls="navMenu" aria-expanded="false" aria-label="Toggle navigation">
      <i class="bi bi-list" aria-hidden="true"></i>
    </button>
    <div id="navMenu" class="hidden w-full lg:flex lg:w-auto lg:items-center lg:gap-8">
      <ul class="flex flex-col py-4 font-medium lg:flex-row lg:gap-6 lg:py-0">
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#home" data-active>Home</a></li>
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#favorites">Favorites</a></li>
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#about">About</a></li>
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#menu">Menu</a></li>
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#reviews">Reviews</a></li>
        <li><a class="block py-2 text-white/75 transition hover:text-white data-active:text-caramel"
href="#contact">Contact</a></li>
      </ul>
      <a class="btn btn-roast mb-4 lg:mb-0" href="#visit">Visit us</a>
    </div>
  </nav>
</header>
```

Wat er wijzigt, en waar

Alles wat in deze paragraaf in de code verandert, per bestand en met de plaats waar het komt. Een `+` voeg je toe, een `-` haal je weg; de regels zonder teken staan er al en tonen waar je zit. De regelnummers zijn die van het bestand ná de wijziging.

`index.html` · gewijzigd

Regel 30–73, in `<style>`:

```
      --font-sans: "Manrope", system-ui, sans-serif;
    }
+
+   /* ---------- 1.2 – De navbar ---------- */
+   … 8 regels, zoals hierboven in deze paragraaf …
+     }
+   }
  </style>
 </head>
 <body class="bg-foam font-sans text-ink antialiased">
+  <header class="sticky top-0 z-50 bg-espresso text-white">
+    <nav id="mainNav" class="mx-auto flex max-w-6xl flex-wrap items-center justify-between px-4 py-3" aria-
label="Main navigation">
+      … 19 regels, zoals hierboven in deze paragraaf …
+  </header>
+
   <main>
-    <div class="mx-auto max-w-6xl px-4 py-12">
-      <h1 class="font-display text-4xl font-bold">Hello, Roast &amp;amp; Co.!</h1>
-      <p class="mt-3 text-lg text-ink/75">If this heading uses a serif font and the button below is caramel,
everything is linked correctly.</p>
-      <button type="button" class="mt-6 rounded-lg bg-caramel-dark px-5 py-2.5 font-semibold text-white
hover:bg-espresso">Test button</button>
-    </div>
+    <!-- Hier komen straks de secties -->
  </main>
 </body>
```

Controleer

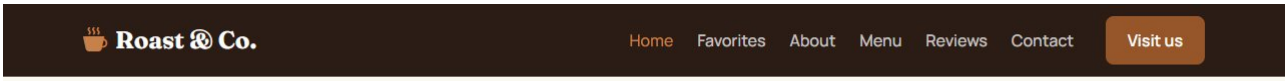
- Op een breed scherm staan de links rechts, met de knop *Visit us* ernaast.
- Op smartphonebreedte verdwijnt het menu en verschijnt de hamburgerknop.
- *Home* is karamelkleurig (`data-active`), de andere links worden wit bij hover.
- Scroll: de balk blijft bovenaan plakken. (Er is nog weinig om te scrollen; dat verandert in 1.3.)

Commit: `git commit -am "Navbar met flexbox en eigen knopcomponent"`.

> **Zelf doen**
>
> Vervang overal `lg:` door `md:` in de navbar. Vanaf welke breedte staat het menu nu naast het logo? Zet het daarna terug.

Documentatie

- Responsive design
- Hover, focus and other states
- Display (flex, hidden)
- Adding custom styles (@apply, @layer)



Resultaat na 1.2

> **Goed om te weten**
>
> **Startpunt en oplossing van 1.2:** `1-begeleid-Roast-and-Co/stap-01-Tailwind CSS I/1.02-De navbar/`, met daarin `startpunt/` en `oplossing/`.

1.3 · De hero

De **hero** is het eerste wat een bezoeker ziet: een grote titel, een korte tekst, twee knoppen en een sfeerfoto. Op een groot scherm staan tekst en foto naast elkaar, op een smartphone onder elkaar.

Nieuw bestand: kopieer `hero.jpg` uit `1-begeleid-Roast-and-Co/afbeeldingen/` naar je map `img/`.

a · Twee kolommen met CSS grid

In Bootstrap verdeelde je een rij in kolommen met `row` en `col-lg-6`. Tailwind heeft geen eigen rooster: je gebruikt rechtstreeks **CSS grid**, met utilities.

> **Weglaten of vervangen**
>
> Vervang de commentaarregel in `<main>` door de hero-sectie.

```
    <main>
-     <!-- Hier komen straks de secties -->
+     <section id="home" class="bg-cream py-16 lg:py-24">
+       <div class="mx-auto grid max-w-6xl items-center gap-10 px-4 lg:grid-cols-2 lg:gap-12">
+         <div>
+           <!-- tekst (b) -->
+         </div>
+         <div class="relative">
+           <!-- foto (c) -->
+         </div>
+       </div>
+     </section>
    </main>
```

Class	Wat doet het?
`id="home"`	Het anker voor de link *Home* in de navbar.
`py-16 lg:py-24`	Padding boven en onder: 4rem op mobiel, 6rem vanaf `lg`.
`grid`	Maakt een CSS grid. Zonder kolommen staat alles **onder elkaar**: perfect voor mobiel.
`lg:grid-cols-2`	Vanaf `lg` twee even brede kolommen. Dit vervangt `row` + twee keer `col-lg-6`.
`items-center`	Centreert de kolommen verticaal.
`gap-10 lg:gap-12`	Ruimte tussen de kolommen, en tussen de rijen op mobiel.

> **Goed om te weten**
>
> **CSS grid in één zin.** Een element met `grid` verdeelt zijn kinderen over cellen. Met `grid-cols-2` (of 3, of 12) bepaal je hoeveel kolommen er zijn; de kinderen vullen de cellen van links naar rechts, rij na rij. Flexbox zet dingen op **één lijn**, grid in **rijen én kolommen**.

De spacing-schaal

Alle afstanden in Tailwind volgen één schaal: **1 = 0.25rem = 4px**.

Class	Waarde	Class	Waarde
`p-1`	0.25rem	`p-6`	1.5rem
`p-2`	0.5rem	`p-8`	2rem
`p-4`	1rem	`p-12`	3rem
`p-5`	1.25rem	`p-24`	6rem

De letter(s) vooraan bepalen de zijde: `p` (alle), `px` (links en rechts), `py` (boven en onder), `pt` `pr` `pb` `pl` (één zijde). Voor marge gebruik je `m` op dezelfde manier: `mb-8`, `mx-auto` …

b · De tekstkolom

```
<div>
  <span class="mb-6 inline-flex items-center gap-2 rounded-full border border-ink/20 px-4 py-1.5 text-sm
font-medium">
    <i class="bi bi-stars text-caramel-dark" aria-hidden="true"></i> New autumn menu
  </span>
  <h1 class="mb-6 font-display text-5xl leading-tight font-bold lg:text-6xl">Coffee that starts <em
class="text-caramel-dark">your story</em></h1>
  <p class="mb-8 text-lg text-ink/75 lg:text-xl">Small-batch beans, roasted in-house every week…</p>
  <div class="mb-12 flex flex-wrap gap-4">
    <a class="btn btn-roast btn-lg" href="#menu">See the menu</a>
    <a class="btn btn-outline btn-lg" href="#visit">
      <i class="bi bi-geo-alt" aria-hidden="true"></i> Find us
    </a>
  </div>
  <!-- cijfers: zie hieronder -->
</div>
```

Class	Wat doet het?
`inline-flex rounded-full border`	Een label (*badge*) met volledig ronde hoeken en een rand.
`text-5xl lg:text-6xl`	Grote titel, nog groter vanaf `lg`.
`leading-tight`	Kleinere regelafstand: grote titels lezen zo beter.
`flex flex-wrap gap-4`	Knoppen naast elkaar; te smal, dan springen ze naar een nieuwe regel.
`btn-lg`, `btn-outline`	Nieuwe knopvarianten (zie d).

> **Goed om te weten**
>
> Er is maar **één** `<h1>` per pagina. Alle andere sectietitels worden `<h2>`.

De cijfers onder de knoppen

Voor "12 single origins" gebruik je een **definitielijst**: `<dt>` is de omschrijving, `<dd>` de waarde. In de HTML komt de omschrijving eerst, maar het getal moet bovenaan staan. Met `flex-col-reverse` draai je de volgorde om **zonder** de HTML te wijzigen:

```
<dl class="grid grid-cols-3 gap-4">
  <div class="flex flex-col-reverse">
    <dt class="text-sm text-ink/70">single origins</dt>
    <dd class="font-display text-3xl font-bold">12</dd>
  </div>
  <!-- nog twee keer hetzelfde -->
</dl>
```

`grid-cols-3`, zonder voorvoegsel, geeft op **elk** scherm drie kolommen.

c · De fotokolom

```
<div class="relative">
  <img src="img/hero.jpg" class="max-h-[560px] w-full rounded-[2rem] object-cover shadow-2xl" alt="A
cappuccino with latte art on a blue saucer" width="900" height="1100">
  <div class="absolute bottom-4 left-4 flex items-center gap-3 rounded-xl bg-white px-4 py-3 shadow-lg
md:bottom-6 md:left-6">
    <span class="grid size-12 place-items-center rounded-full bg-cream text-xl text-caramel-dark"><i
class="bi bi-fire" aria-hidden="true"></i></span>
    <div>
      <p class="font-bold">Fresh roast</p>
      <p class="text-sm text-ink/70">every Monday morning</p>
    </div>
  </div>
</div>
```

Class	Wat doet het?
`max-h-[560px]`	Een **arbitrary value**: staat de waarde die je nodig hebt niet in de schaal, zet ze dan tussen `[ ]`.
`rounded-[2rem]`	Ook een arbitrary value: een grotere afronding dan `rounded-3xl`.
`w-full object-cover`	De foto vult de breedte en wordt bijgesneden in plaats van vervormd.
`relative` / `absolute bottom-4 left-4`	Het kaartje zweeft linksonder over de foto: `absolute` plaatst het ten opzichte van de dichtstbijzijnde ouder met `relative`.
`size-12`	Breedte **en** hoogte 3rem in één class.
`grid place-items-center`	Centreert het icoon horizontaal én verticaal.

> **Tip**
>
> Zet `width` en `height` op een `<img>`. Dan reserveert de browser al plaats voor de foto geladen is, en springt je pagina niet.

d · Twee extra knopvarianten

Voeg toe aan je `<style type="text/tailwindcss">`:

```
@layer components {
  .btn-lg {
    @apply px-7 py-3.5 text-lg;
  }

  .btn-outline {
    @apply border-ink text-ink hover:bg-ink hover:text-white;
  }
}
```

Een knop combineert nu altijd `btn` (vorm) + een kleur (`btn-roast` of `btn-outline`) + optioneel een grootte (`btn-lg`). Hetzelfde idee als `btn btn-primary btn-lg` in Bootstrap, maar nu met je eigen classes.

De volledige hero

```
<section id="home" class="bg-cream py-16 lg:py-24">
  <div class="mx-auto grid max-w-6xl items-center gap-10 px-4 lg:grid-cols-2 lg:gap-12">
    <div>
      <span class="mb-6 inline-flex items-center gap-2 rounded-full border border-ink/20 px-4 py-1.5 text-sm
font-medium">
```

```
        <i class="bi bi-stars text-caramel-dark" aria-hidden="true"></i> New autumn menu
      </span>
      <h1 class="mb-6 font-display text-5xl leading-tight font-bold lg:text-6xl">Coffee that starts <em
class="text-caramel-dark">your story</em></h1>
      <p class="mb-8 text-lg text-ink/75 lg:text-xl">Small-batch beans, roasted in-house every week and
brewed with care. Stay for a slow morning or grab your favourite on the go.</p>
      <div class="mb-12 flex flex-wrap gap-4">
        <a class="btn btn-roast btn-lg" href="#menu">See the menu</a>
        <a class="btn btn-outline btn-lg" href="#visit">
          <i class="bi bi-geo-alt" aria-hidden="true"></i> Find us
        </a>
      </div>
      <dl class="grid grid-cols-3 gap-4">
        <div class="flex flex-col-reverse">
          <dt class="text-sm text-ink/70">single origins</dt>
          <dd class="font-display text-3xl font-bold">12</dd>
        </div>
        <div class="flex flex-col-reverse">
          <dt class="text-sm text-ink/70">Google rating</dt>
          <dd class="font-display text-3xl font-bold">4.9</dd>
        </div>
        <div class="flex flex-col-reverse">
          <dt class="text-sm text-ink/70">roasting since</dt>
          <dd class="font-display text-3xl font-bold">2015</dd>
        </div>
      </dl>
    </div>
    <div class="relative">
      <img src="img/hero.jpg" class="max-h-[560px] w-full rounded-[2rem] object-cover shadow-2xl" alt="A
cappuccino with latte art on a blue saucer" width="900" height="1100">
      <div class="absolute bottom-4 left-4 flex items-center gap-3 rounded-xl bg-white px-4 py-3 shadow-lg
md:bottom-6 md:left-6">
        <span class="grid size-12 place-items-center rounded-full bg-cream text-xl text-caramel-dark"><i
class="bi bi-fire" aria-hidden="true"></i></span>
        <div>
          <p class="font-bold">Fresh roast</p>
          <p class="text-sm text-ink/70">every Monday morning</p>
        </div>
      </div>
    </div>
  </div>
</section>
```

Wat er wijzigt, en waar

Alles wat in deze paragraaf in de code verandert, per bestand en met de plaats waar het komt. Een `+` voeg je toe, een `-` haal je weg; de regels zonder teken staan er al en tonen waar je zit. De regelnummers zijn die van het bestand ná de wijziging.

`index.html` · gewijzigd

Regel 42–56, in `<style>`:

```
      }
    }
+
+   /* ---------- 1.3 – De hero ---------- */
+   @layer components {
+     .btn-lg {
+       @apply px-7 py-3.5 text-lg;
+     }
+
+     .btn-outline {
+       @apply border-ink text-ink hover:bg-ink hover:text-white;
+     }
+   }
  </style>
 </head>
```

Regel 80–124, in `<main>`:

```
   <main>
-    <!-- Hier komen straks de secties -->
+    <section id="home" class="bg-cream py-16 lg:py-24">
+      <div class="mx-auto grid max-w-6xl items-center gap-10 px-4 lg:grid-cols-2 lg:gap-12">
+        … 37 regels, zoals hierboven in deze paragraaf …
+      </div>
+    </section>
   </main>
 </body>
```

Nieuwe afbeeldingen, uit de map `afbeeldingen/`: `img/hero.jpg`.

Controleer

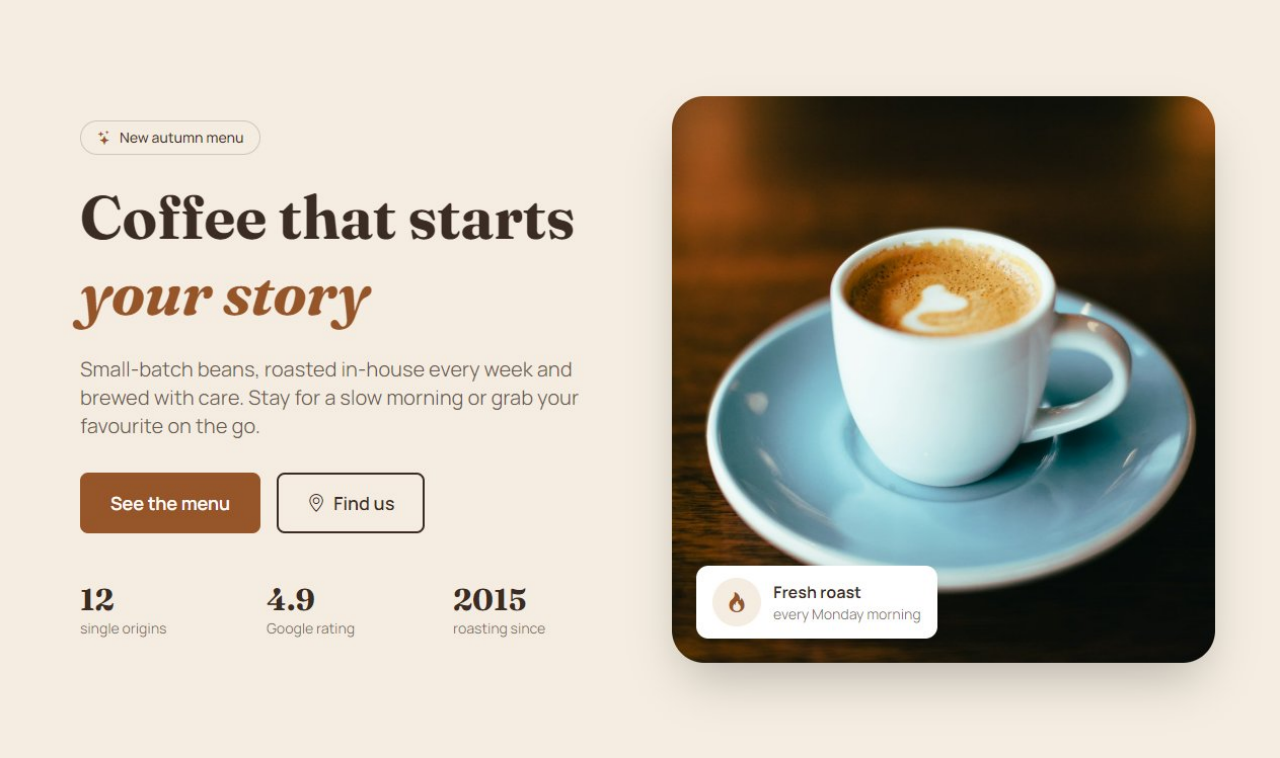
- Groot scherm: tekst links, foto rechts, verticaal gecentreerd.
- Smartphone: tekst boven, foto eronder; de knoppen springen eventueel onder elkaar.
- Geen horizontale scrollbalk op smartphonebreedte.

Commit: `git commit -am "Hero met grid en arbitrary values"`. Vergeet `git add img/` niet: de foto is een nieuw bestand.

> **Zelf doen**
>
> Wissel `lg:grid-cols-2` voor `md:grid-cols-2`. Vanaf welke breedte staan tekst en foto nu naast elkaar? Zet het daarna terug.

Documentatie

- Grid template columns
- Padding · Margin
- Using arbitrary values
- Position · Object fit



Resultaat na 1.3

> **Goed om te weten**
>
> **Startpunt en oplossing van 1.3:** `1-begeleid-Roast-and-Co/stap-01-Tailwind CSS I/1.03-De hero/`, met daarin `startpunt/` en `oplossing/`.

1.4 · Populaire koffies

Drie **productkaarten** met foto, naam, label, beschrijving en prijs. Op een smartphone staan ze onder elkaar, op een tablet per twee, op een laptop per drie.

Nieuwe bestanden: `espresso.jpg`, `cappuccino.jpg` en `iced-latte.jpg` naar `img/`.

a · Sectie en titel

Zet de nieuwe sectie **onder** de hero, nog binnen `<main>`.

```
<section id="favorites" class="py-16 lg:py-24">
  <div class="mx-auto max-w-6xl px-4">
    <div class="mb-12 flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-between">
      <div class="lg:max-w-xl">
        <p class="eyebrow">Our favourites</p>
        <h2 class="font-display text-4xl font-bold lg:text-5xl">Three cups our regulars love</h2>
      </div>
      <p class="text-ink/75 lg:max-w-sm">Every drink starts with our house espresso blend…</p>
    </div>

    <!-- kaarten (b) -->
  </div>
</section>
```

Class	Wat doet het?
`flex flex-col lg:flex-row`	Titel en tekst onder elkaar op mobiel, naast elkaar vanaf `lg`.
`lg:items-end lg:justify-between`	Vanaf `lg`: onderaan uitgelijnd, titel links en tekst rechts.
`lg:max-w-xl`, `lg:max-w-sm`	Maximale breedtes, zodat regels niet te lang worden.
`eyebrow`	Eigen component: een klein label in hoofdletters boven de titel (zie d).

b · Een responsive grid

```
<div class="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
  <article> kaart 1 </article>
  <article> kaart 2 </article>
  <article> kaart 3 </article>
</div>
```

Class	Resultaat
`grid`	één kaart per rij (smartphone)
`md:grid-cols-2`	twee per rij vanaf 768px
`lg:grid-cols-3`	drie per rij vanaf 1024px
`gap-6`	1.5rem ruimte tussen de kaarten

Vergelijk met Bootstrap: daar schreef je `row row-cols-1 row-cols-md-2 row-cols-lg-3`. Het idee is hetzelfde, de classes zijn anders.

> **Goed om te weten**
>
> Items in een grid zijn standaard **even hoog** per rij. Daar hoef je niets voor te doen.

c · Een kaart opbouwen

Tailwind heeft geen `card`. Een kaart is gewoon een `<article>` met een witte achtergrond, afgeronde hoeken en een schaduw. Een eerste versie:

```
<article class="rounded-2xl bg-white shadow-sm">
  <img src="img/espresso.jpg" class="h-48 w-full object-cover" alt="A cup of espresso seen from above">
  <div class="p-6">
    <h3 class="font-display text-2xl font-bold">Classic Espresso</h3>
    <p class="text-ink/75">Short, bold and full of flavour…</p>
    <span class="text-2xl font-bold">€2,80</span>
  </div>
</article>
```

Die werk je af tot de uiteindelijke productkaart:

```
- <article class="rounded-2xl bg-white shadow-sm">
-   <img src="img/espresso.jpg" class="h-48 w-full object-cover" alt="A cup of espresso seen from above">
-   <div class="p-6">
-     <h3 class="font-display text-2xl font-bold">Classic Espresso</h3>
+ <article class="group flex flex-col overflow-hidden rounded-2xl bg-white shadow-sm transition duration-300
hover:-translate-y-1.5 hover:shadow-xl motion-reduce:transition-none motion-reduce:hover:translate-y-0">
+   <div class="overflow-hidden">
+     <img src="img/espresso.jpg" class="aspect-4/3 w-full object-cover transition duration-500 group-
hover:scale-105" alt="A cup of espresso seen from above" loading="lazy">
+   </div>
+   <div class="flex flex-1 flex-col p-6">
+     <div class="mb-2 flex items-start justify-between gap-2">
+       <h3 class="font-display text-2xl font-bold">Classic Espresso</h3>
+       <span class="rounded-full bg-espresso px-2.5 py-1 text-xs font-bold text-white">Bestseller</span>
+     </div>
      <p class="text-ink/75">Short, bold and full of flavour…</p>
-     <span class="text-2xl font-bold">€2,80</span>
+     <div class="mt-auto flex items-center justify-between pt-6">
+       <span class="text-2xl font-bold">€2,80</span>
+       <a href="#menu" class="rounded-full border border-ink px-4 py-1.5 text-sm font-semibold transition
hover:bg-ink hover:text-white">Order</a>
+     </div>
    </div>
  </article>
```

Weglaten of vervangen

- `h-48` (een vaste hoogte) gaat **weg**. In de plaats komt `aspect-4/3`: elke foto krijgt dezelfde verhouding 4:3, op elke schermbreedte.
- `overflow-hidden` op de kaart zorgt dat de foto de afgeronde hoeken volgt.

Class	Wat doet het?
`flex flex-col` + `flex-1` + `mt-auto`	De kaart is een kolom; de inhoud rekt uit en `mt-auto` duwt de prijs **naar onderen**. Zo staan alle prijzen op dezelfde hoogte.
`group` / `group-hover:scale-105`	Beweeg je over de **kaart** (de *group*), dan zoomt de **foto** in.
`hover:-translate-y-1.5`	De kaart schuift 0.375rem omhoog. Een `-` vooraan maakt de waarde negatief.
`transition duration-300`	Een vloeiende overgang van 300 ms.
`motion-reduce:…`	Geen beweging als de gebruiker animaties uitgeschakeld heeft.
`loading="lazy"`	De browser laadt de foto pas als je in de buurt scrolt.

Maak nu nog twee kaarten: **Velvet Cappuccino** (label `bg-amber-400 text-espresso` *Signature*, €3,90) en **Iced Caramel Latte** (label `bg-emerald-700 text-white` *New*, €4,60). `amber` en `emerald` komen uit het **standaardpalet** van Tailwind; dat kan je naast je eigen kleuren gewoon blijven gebruiken.

d · De eyebrow-component

```
@layer components {
  /* Klein label boven een sectietitel */
  .eyebrow {
    @apply mb-3 text-xs font-bold tracking-[0.12em] text-caramel-dark uppercase;
  }
}
```

`tracking-[0.12em]` is weer een arbitrary value: iets meer letterafstand dan `tracking-widest`.

De volledige sectie

```
<section id="favorites" class="py-16 lg:py-24">
  <div class="mx-auto max-w-6xl px-4">
    <div class="mb-12 flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-between">
      <div class="lg:max-w-xl">
        <p class="eyebrow">Our favourites</p>
        <h2 class="font-display text-4xl font-bold lg:text-5xl">Three cups our regulars love</h2>
      </div>
      <p class="text-ink/75 lg:max-w-sm">Every drink starts with our house espresso blend: chocolatey, round
and roasted just a few days ago.</p>
    </div>

    <div class="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
      <article class="group flex flex-col overflow-hidden rounded-2xl bg-white shadow-sm transition duration-
300 hover:-translate-y-1.5 hover:shadow-xl motion-reduce:transition-none motion-reduce:hover:translate-y-0">
        <div class="overflow-hidden">
          <img src="img/espresso.jpg" class="aspect-4/3 w-full object-cover transition duration-500 group-
hover:scale-105" alt="A cup of espresso seen from above" loading="lazy">
        </div>
        <div class="flex flex-1 flex-col p-6">
          <div class="mb-2 flex items-start justify-between gap-2">
            <h3 class="font-display text-2xl font-bold">Classic Espresso</h3>
            <span class="rounded-full bg-espresso px-2.5 py-1 text-xs font-bold text-white">Bestseller</span>
          </div>
          <p class="text-ink/75">Short, bold and full of flavour. Notes of dark chocolate and roasted
hazelnut.</p>
          <div class="mt-auto flex items-center justify-between pt-6">
            <span class="text-2xl font-bold">€2,80</span>
            <a href="#menu" class="rounded-full border border-ink px-4 py-1.5 text-sm font-semibold
transition hover:bg-ink hover:text-white">Order</a>
          </div>
        </div>
      </article>

      <article class="group flex flex-col overflow-hidden rounded-2xl bg-white shadow-sm transition duration-
300 hover:-translate-y-1.5 hover:shadow-xl motion-reduce:transition-none motion-reduce:hover:translate-y-0">
        <div class="overflow-hidden">
          <img src="img/cappuccino.jpg" class="aspect-4/3 w-full object-cover transition duration-500 group-
hover:scale-105" alt="A cappuccino with a latte art leaf" loading="lazy">
        </div>
        <div class="flex flex-1 flex-col p-6">
          <div class="mb-2 flex items-start justify-between gap-2">
            <h3 class="font-display text-2xl font-bold">Velvet Cappuccino</h3>
            <span class="rounded-full bg-amber-400 px-2.5 py-1 text-xs font-bold text-
espresso">Signature</span>
          </div>
          <p class="text-ink/75">A double shot with silky milk foam. Ask for oat milk at no extra cost.</p>
          <div class="mt-auto flex items-center justify-between pt-6">
            <span class="text-2xl font-bold">€3,90</span>
            <a href="#menu" class="rounded-full border border-ink px-4 py-1.5 text-sm font-semibold
transition hover:bg-ink hover:text-white">Order</a>
          </div>
        </div>
      </article>

      <article class="group flex flex-col overflow-hidden rounded-2xl bg-white shadow-sm transition duration-
300 hover:-translate-y-1.5 hover:shadow-xl motion-reduce:transition-none motion-reduce:hover:translate-y-0">
        <div class="overflow-hidden">
```

```
          <img src="img/iced-latte.jpg" class="aspect-4/3 w-full object-cover transition duration-500 group-
hover:scale-105" alt="An iced caramel latte in a tall glass" loading="lazy">
        </div>
        <div class="flex flex-1 flex-col p-6">
          <div class="mb-2 flex items-start justify-between gap-2">
            <h3 class="font-display text-2xl font-bold">Iced Caramel Latte</h3>
            <span class="rounded-full bg-emerald-700 px-2.5 py-1 text-xs font-bold text-white">New</span>
          </div>
          <p class="text-ink/75">Cold milk, espresso and our homemade salted caramel over ice.</p>
          <div class="mt-auto flex items-center justify-between pt-6">
            <span class="text-2xl font-bold">€4,60</span>
            <a href="#menu" class="rounded-full border border-ink px-4 py-1.5 text-sm font-semibold
transition hover:bg-ink hover:text-white">Order</a>
          </div>
        </div>
      </article>
    </div>
  </div>
</section>
```

Wat er wijzigt, en waar

Alles wat in deze paragraaf in de code verandert, per bestand en met de plaats waar het komt. Een `+` voeg je toe, een `-` haal je weg; de regels zonder teken staan er al en tonen waar je zit. De regelnummers zijn die van het bestand ná de wijziging.

`index.html` · gewijzigd

Regel 53–64, in `<style>`:

```
       }
     }
+
+    /* ---------- 1.4 – Populaire koffies ---------- */
+    @layer components {
+      /* Klein label boven een sectietitel */
+      .eyebrow {
+        @apply mb-3 text-xs font-bold tracking-[0.12em] text-caramel-dark uppercase;
+      }
+    }
   </style>
 </head>
```

Regel 129–197, in `<main>`:

```
       </div>
     </section>
+
+    <section id="favorites" class="py-16 lg:py-24">
+      … 61 regels, zoals hierboven in deze paragraaf …
+      </div>
+    </section>
   </main>
 </body>
```

Nieuwe afbeeldingen, uit de map `afbeeldingen/`: `img/espresso.jpg`, `img/cappuccino.jpg`, `img/iced-latte.jpg`.

Controleer

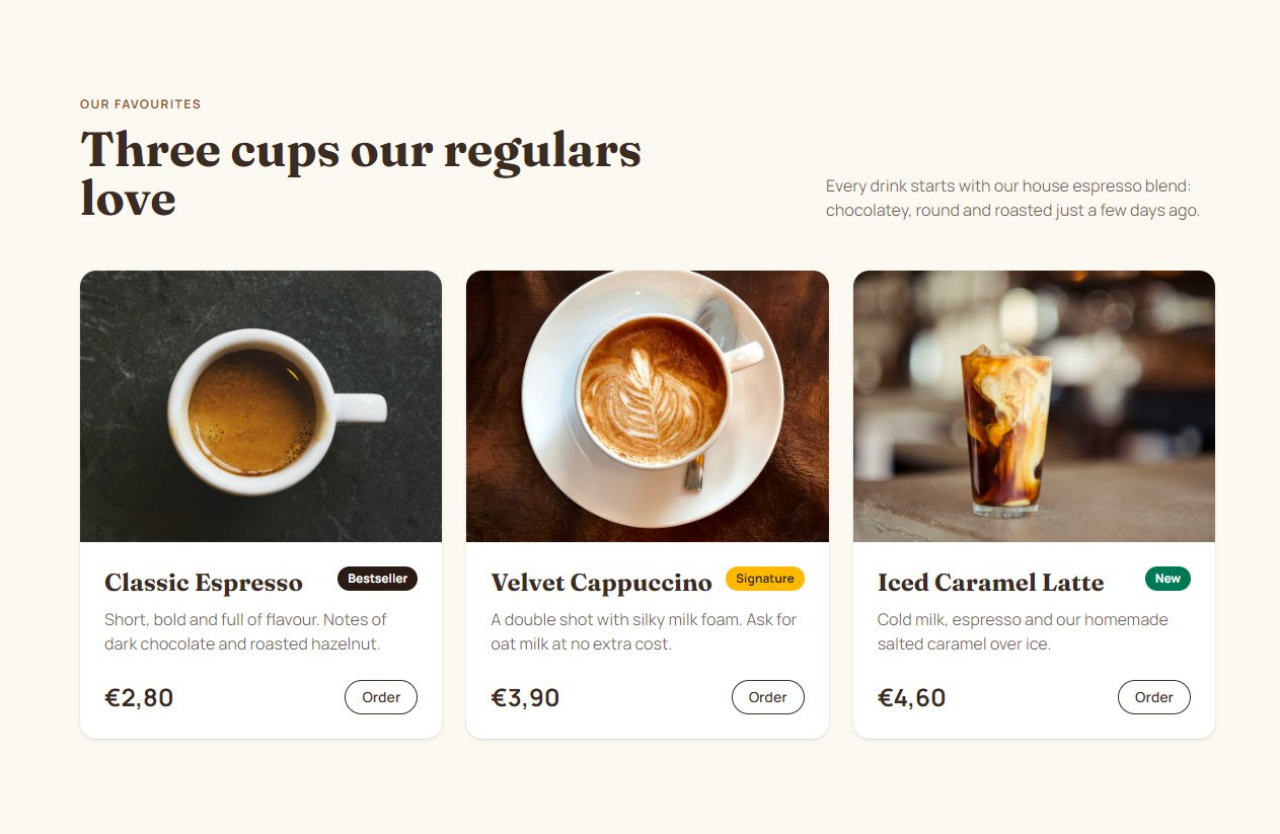
- Drie kaarten naast elkaar op een breed scherm, twee op een tablet, één op een smartphone.
- Alle foto's hebben dezelfde verhouding, alle kaarten dezelfde hoogte, alle prijzen staan op één lijn.
- Beweeg over een kaart: ze komt omhoog en de foto zoomt in.

Commit: `git add .` en `git commit -m "Populaire koffies in een responsive grid"`. Zet je repository daarna op GitHub, zoals in module 1.

> **Zelf doen**
>
> Voeg een **vierde** koffie toe. Wat gebeurt er met de indeling op een laptop? Probeer daarna `lg:grid-cols-4` en kijk wat er met de kaarten gebeurt.

Documentatie

- Grid template columns
- Aspect ratio
- Styling based on parent state (group)
- Colors (standaardpalet)



Resultaat na 1.4

> **Goed om te weten**
>
> **Startpunt en oplossing van 1.4:** `1-begeleid-Roast-and-Co/stap-01-Tailwind CSS I/1.04-Populaire koffies/`, met daarin `startpunt/` en `oplossing/`.

1.5 · Oefening 1 — Het startproject van Sel & Poivre

Rechtstreeks toepassen · leerstof uit 1.1 · oplossing: Oplossing 1.5

De situatie

Sel & Poivre is een kookatelier in Gent. Chefs geven er workshops in kleine groepen: brood bakken, verse pasta, sushi, chocolade. Het atelier wil een eigen website, in het Nederlands, met een warme maar frisse huisstijl. Jij bouwt die site, naast Roast & Co., de hele module lang.

Wat je maakt

Een nieuw project `sel-et-poivre`, opgezet zoals Roast & Co. in 1.1, maar met de huisstijl van het kookatelier.

1. Maak de map `sel-et-poivre` met een `index.html` en een lege map `img/`. Maak er een Git-repository van.
2. Het HTML-skelet is **Nederlandstalig**: kies de juiste waarde voor `lang`. Titel: *Sel & Poivre – kookatelier in Gent*. Schrijf zelf een beschrijving van één zin voor de `description`.
3. Koppel Tailwind 4.3.3 via de Play CDN en Bootstrap Icons 1.13.1.
4. Koppel deze lettertypes van Google Fonts:

```
https://fonts.googleapis.com/css2?family=DM+Serif+Display:ital@0;1&family=Source+Sans+3:wght@400;600;700&display=swap
```

5. Leg de huisstijl vast in `@theme`:

Naam	Waarde	Gebruik
`basil`	`#2f5d50`	hoofdkleur, groen
`basil-dark`	`#1f3f36`	donkere variant
`tomato`	`#d0532f`	accent, alleen voor iconen en grote tekst
`tomato-dark`	`#a63d20`	knoppen en accenten in tekst
`flour`	`#fbf8f2`	achtergrond van de pagina
`linen`	`#f0e8da`	achtergrond van secties
`pepper`	`#22262a`	tekstkleur

   Titels in *DM Serif Display*, gewone tekst in *Source Sans 3*.
6. Geef de `<body>` de achtergrond `flour`, de tekstkleur `pepper` en het tekstlettertype.
7. Zet een testblok in `<main>`: een titel, een korte zin en een tomatenrode knop die groen wordt bij hover. Gebruik als container **`max-w-5xl`** in plaats van `max-w-6xl`: Sel & Poivre is een smallere site dan Roast & Co.

Zo controleer je

- De titel staat in een schreefletter, de tekst niet.
- De knop is tomatenrood en wordt basilicumgroen bij hover.
- De achtergrond is gebroken wit, niet zuiver wit.